Why does this matter?

- What does it mean to say that an operational semantics or implementation is correct?
- How can we reason about the validity of program optimisations?

```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```

Can we release cowns that are highly contended?

Can we reorder behaviours?

How do we address it

- We could develop a more involved operational semantics?
- More complicated, may still end up being overly restrictive or permissive without a means to measure that quality (there feels like there should be something more there)
- Can we take inspiration from the way others have formulated these designs, namely weak memory models